

Relatório de Testes

Nodify — Sistema de Acessibilidade Cefálica para Spotify

Data: 25/09/2026 Casos de uso: UC01, UC02, UC03 Plataforma: Android (Kotlin + Jetpack Compose)

4. TESTES

Este relatório consolida os testes automatizados do Nodify, organizados em três níveis complementares, e a avaliação heurística da interface. Ao todo foram implementados **270 testes automatizados**: 186 unitários, 33 de integração e 51 funcionais de interface.

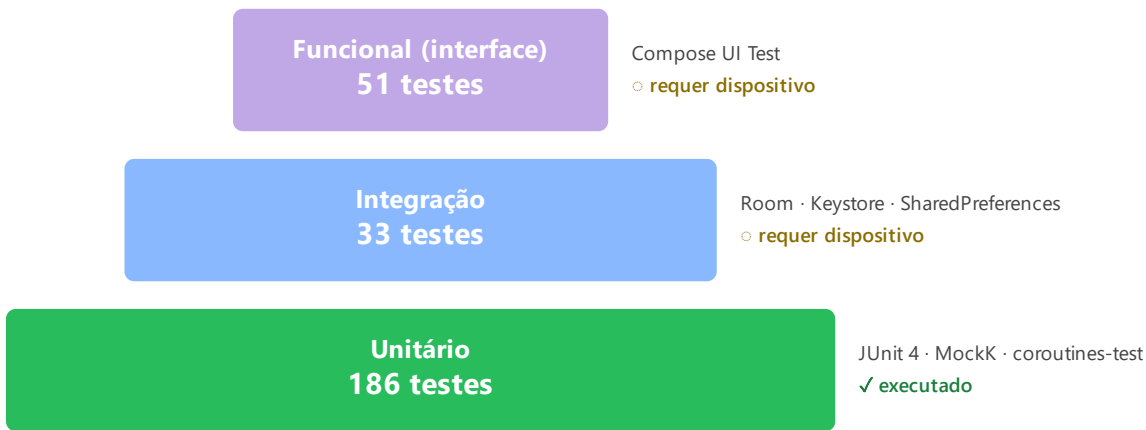


Figura 1 — Distribuição dos testes automatizados por nível.

Situação de execução. Os **186 testes unitários** foram executados nesta data e passaram integralmente (0 falhas). Os **84 testes instrumentados** (integração e funcionais de interface) estão implementados e compilam, mas **ainda não foram executados**: exigem aparelho Android ou emulador conectado, indisponível na máquina de desenvolvimento no momento da geração deste relatório. As linhas correspondentes estão marcadas como **Pendente** — nenhum resultado foi presumido. Para executá-los: `./gradlew connectedDebugAndroidTest`.

4.1 Teste funcional

Verifique se as funcionalidades implementadas atendem aos requisitos especificados. Os testes devem considerar a aplicação como caixa-preta, observando entradas, ações e resultados.

Nota sobre a coluna “Requisito”. O projeto especifica seus requisitos como **casos de uso** (UC01 — Controlar reprodução via gestos; UC02 — Calibrar sensibilidade e movimentos; UC03 — Customizar mapeamento de comandos), e não como histórias de usuário. A coluna foi preenchida com o caso de uso correspondente.

Nota sobre o nível de execução. Os casos TF01–TF21 são verificados no nível de componente: exercitam o comportamento observável a partir das entradas do usuário (gesto detectado, toque, resposta da API), sem inspecionar a implementação interna. Os casos TF22–TF24 são verificados no nível da regra de decisão — a política de qualidade da captura roda na JVM, enquanto a exibição do aviso na tela depende de dispositivo. Os casos TF25–TF27 percorrem a interface renderizada.

ID	Requisito	Cenário de teste	Resultado esperado	Resultado obtido	Situação
TF01	UC01	Usuário conclui o login e o app recebe o código de autorização pelo deep link <code>sac://callback</code> .	Sessão autenticada e a tela avança.	Estado <code>Success</code> após a emissão do código.	Aprovado
TF02	UC01	A troca do código por token falha (<code>invalid_grant</code>).	Mensagem de erro exibida, sem travar a tela.	Estado <code>Error("invalid_grant")</code> ; ao limpar, volta a <code>Idle</code> .	Aprovado
TF03	UC01	Com a música tocando, o usuário executa o gesto "Aceno (sim)".	A reprodução é pausada.	<code>pause()</code> chamado 1×; <code>play()</code> 0×.	Aprovado
TF04	UC01	Com a música pausada, o usuário executa o gesto "Aceno (sim)".	A reprodução inicia.	<code>play()</code> chamado 1×; <code>pause()</code> 0×.	Aprovado
TF05	UC01	O usuário executa "Virar rosto para a direita".	Avança para a próxima faixa.	<code>skipToNext()</code> chamado 1×.	Aprovado
TF06	UC01	O usuário executa "Virar rosto para a esquerda".	Volta para a faixa anterior.	<code>skipToPrevious()</code> chamado 1×.	Aprovado
TF07	UC01	O usuário executa um gesto sem ação mapeada (piscar olho esquerdo, no padrão).	Nenhum comando é enviado ao Spotify.	0 chamadas a <code>play/pause/next/previous/volume</code> ; sem indicador na tela.	Aprovado
TF08	UC01	Volume em 100% e o usuário executa "Inclinar para a direita".	O volume permanece em 100%, sem erro.	<code>volumePercent = 100</code> ; nenhuma chamada com valor > 100.	Aprovado
TF09	UC01	Volume em 0% e o usuário executa "Inclinar para a esquerda".	O volume permanece em 0%, sem erro.	<code>volumePercent = 0</code> ; nenhuma chamada com valor < 0.	Aprovado
TF10	UC01	Um gesto é reconhecido durante o rastreamento.	Indicador visual mostra o gesto e a ação por 2 s.	Indicador ativa e desativa exatamente após 2000 ms.	Aprovado
TF11	UC01	Comando enviado por conta sem Premium (HTTP 403).	Mensagem explicando a exigência de Premium.	"Spotify Premium necessário para controle de reprodução".	Aprovado
TF12	UC01	Comando enviado sem dispositivo Spotify ativo (HTTP 404).	Mensagem orientando abrir o Spotify.	"Nenhum dispositivo ativo. Abra o Spotify no celular primeiro."	Aprovado
TF13	UC01	Nenhuma faixa em reprodução (HTTP 204).	A tela informa que nada está tocando.	<code>trackTitle = "Nada tocando"</code> ; sem capa e sem artista.	Aprovado
TF14	UC02	A calibração conclui com pico de 40° no eixo roll.	Limiar definido em 75% do pico medido.	30,0°.	Aprovado
TF15	UC02	O movimento medido na calibração é pequeno demais (4°).	O mínimo de segurança do eixo é aplicado.	6,0° (mínimo de roll), evitando falso positivo.	Aprovado
TF16	UC02	Câmera com convenção de sinais invertida.	A polaridade aprendida corrige direita/esquerda.	<code>rollSign = -1</code> ; magnitudes preservadas.	Aprovado

ID	Requisito	Cenário de teste	Resultado esperado	Resultado obtido	Situação
TF17	UC03	O usuário abre a tela “Configurar Gestos”.	Os 9 gestos são listados com a ação atual de cada um.	9 linhas na ordem do vocabulário, com rótulos corretos.	Aprovado
TF18	UC03	O usuário troca a ação de um gesto e salva.	O novo mapeamento é persistido.	Mapa gravado contém a ação alterada; as demais linhas intactas.	Aprovado
TF19	UC03	O usuário marca um gesto como “(Sem ação)”.	O gesto é salvo sem ação associada.	Chave presente no mapa com valor nulo.	Aprovado
TF20	UC03	O usuário aciona “Restaurar padrões”.	O mapeamento volta à configuração padrão.	Operação delegada ao repositório; 9 gestos redefinidos.	Aprovado
TF21	UC03	O usuário remapeia um gesto com o rastreamento já em uso.	O gesto passa a disparar o novo comando imediatamente.	Após remapear o aceno, dispara “próxima faixa” em vez de “tocar”.	Aprovado
TF22	UC02	O usuário inicia a calibração num ambiente escuro.	A mensagem orienta acender a luz, em vez de culpar o posicionamento.	Classificado como “ambiente escuro” mesmo sem rosto detectado — a luz tem prioridade.	Aprovado
TF23	UC02	Uma segunda pessoa aparece no quadro durante a calibração.	A calibração é bloqueada e avisa que só um rosto deve aparecer.	Classificado como “mais de um rosto” e marcado como bloqueante.	Aprovado
TF24	UC02	O usuário fica distante demais ou colado na câmera.	A mensagem pede para aproximar ou afastar.	“Longe” abaixo de 22 % e “perto” acima de 85 % da altura do quadro.	Aprovado
TF25	UC03	(interface) Tocar no seletor de ação e escolher outra opção na tela real.	A tela notifica o gesto e o rótulo corretos.	—	Pendente
TF26	UC01	(interface) Acionar os cinco controles de reprodução na tela real.	Cada botão dispara o comando correspondente.	—	Pendente
TF27	UC01/UC03	(interface) Varredura dos elementos clicáveis das 4 telas.	Todos têm rótulo para TalkBack e alvo ≥ 48 dp.	—	Pendente

Indicadores — teste funcional

Indicador	Resultado
Quantidade total de testes	27
Testes aprovados	24
Testes reprovados	0
Testes pendentes de execução	3
Taxa de sucesso (sobre os executados)	100% (24/24)

Test Summary

186

tests

0

failures

0

ignored

4.477s

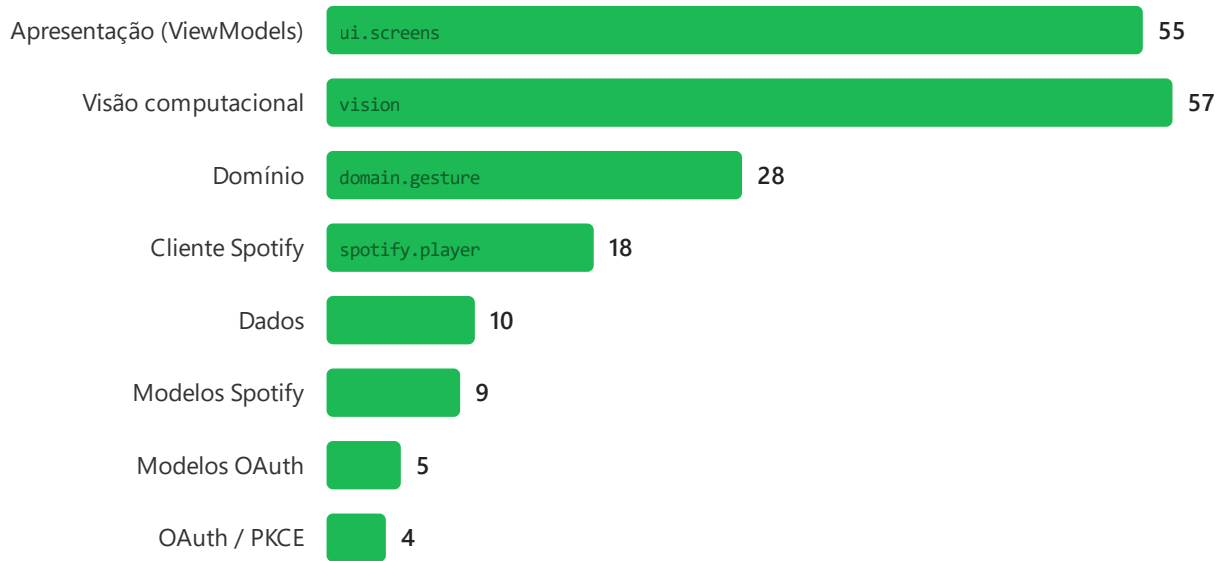
duration

100%

successful

Packages		Classes			
Package	Tests	Failures	Ignored	Duration	Success
com.sac.acessibilidade.data.gesture	10	0	0	2.065s	100%
com.sac.acessibilidade.domain.gesture	28	0	0	0.010s	100%
com.sac.acessibilidade.spotify.auth	4	0	0	0.003s	100%
com.sac.acessibilidade.spotify.auth.model	5	0	0	0s	100%
com.sac.acessibilidade.spotify.player	18	0	0	0.246s	100%
com.sac.acessibilidade.spotify.player.model	9	0	0	0s	100%
com.sac.acessibilidade.ui.screens	55	0	0	0.710s	100%
com.sac.acessibilidade.vision	57	0	0	1.443s	100%

Figura 2 — Relatório gerado pelo Gradle: 186 testes unitários, 0 falhas, 100% de sucesso.



Total: 186 testes unitários — 0 falhas

Figura 3 — Testes unitários executados, por pacote da aplicação.

4.2 Teste de integração

4.2.1 Integração com banco de dados

Demonstre que o sistema interage corretamente com o banco de dados, considerando as operações de criação, leitura, atualização e exclusão, quando aplicáveis.

O Nodify persiste dois conjuntos de dados: o **mapeamento gesto→ação** (SQLite via Room) e os **parâmetros de calibração** (SharedPreferences). Os tokens de autenticação ficam em armazenamento cifrado pelo Android Keystore. Os testes abaixo instanciam as classes reais — banco em memória e preferências do próprio aparelho — sem simulação.

ID	Integração testada	Operação	Resultado esperado	Resultado obtido
TI01	Aplicação ↔ Banco (Room/SQLite)	Inserção	Gravar dois mapeamentos e recuperá-los com gesto e ação corretos.	Pendente
TI02	Aplicação ↔ Banco (Room/SQLite)	Consulta	Com a tabela vazia, o repositório entrega os 9 mapeamentos padrão.	Pendente
TI03	Aplicação ↔ Banco (Room/SQLite)	Atualização	Regravar o mesmo gesto substitui a linha, sem duplicar (o gesto é chave primária).	Pendente
TI04	Aplicação ↔ Banco (Room/SQLite)	Exclusão	“Restaurar padrões” limpa a tabela antes de regravar, sem deixar linhas órfãs.	Pendente
TI05	Aplicação ↔ SharedPreferences	Inserção / Consulta	Round-trip dos 11 limiares de calibração com valores distintos, incluindo polaridade negativa.	Pendente
TI06	Aplicação ↔ Android Keystore	Inserção / Exclusão	Tokens gravados cifrados; “limpar tudo” remove tokens, verificador PKCE e <i>state</i> .	Pendente

Embora a integração com o banco real dependa de dispositivo, a **lógica do repositório** que opera sobre ele foi validada na JVM com o acesso a dados simulado: 10 testes aprovados, cobrindo a entrega dos padrões quando a tabela está vazia, a conversão entre registros e tipos do domínio, a gravação de gestos sem ação e a ordem correta das operações em “Restaurar padrões” (limpar antes de regravar — a ordem inversa deixaria o usuário sem nenhum gesto ativo).

4.2.2 Integração com APIs e serviços

Caso o projeto utilize APIs, serviços externos ou outros componentes, apresente os testes realizados.

O Nodify integra-se com a **Spotify Web API** (controle de reprodução e leitura da faixa atual) e com o **Spotify Accounts Service** (OAuth 2.0 com PKCE). Nos casos TI07–TI13 a resposta HTTP é simulada, o que permite exercitar de forma determinística os códigos de status que seriam difíceis de provocar em rede real — 403 sem Premium, 404 sem dispositivo ativo, 500 do servidor.

ID	Componente	Integração	Resultado esperado	Resultado obtido	Situação
TI07	Spotify Web API	PUT /me/player/play e /pause	HTTP 204 é traduzido em sucesso de domínio.	Sucesso; endpoint correto por comando.	Aprovado
TI08	Spotify Web API	POST /me/player/next e /previous	HTTP 204 é traduzido em sucesso de domínio.	Sucesso; endpoint correto por comando.	Aprovado

ID	Componente	Integração	Resultado esperado	Resultado obtido	Situação
T109	Spotify Web API	PUT /me/player/volume	O valor é saturado na faixa 0–100 antes do envio.	150 → 100; –5 → 0; 45 → 45.	Aprovado
T110	Spotify Web API	GET /me/player/currently-playing	204 indica “nada tocando”; 200 devolve a faixa desserializada.	204 → nulo com sucesso; 200 → título, artista e capa.	Aprovado
T111	Spotify Web API	GET /me	Perfil desserializado; corpo vazio vira erro explícito.	Perfil correto; corpo nulo → “Perfil vazio”.	Aprovado
T112	Spotify Web API	Tratamento de erros 403 / 404 / 500	Cada código vira uma mensagem de domínio específica.	403 e 404 com texto orientativo; demais como “HTTP n”.	Aprovado
T113	Spotify Accounts (OAuth 2.0)	PKCE — RFC 7636	Desafio = base64url(SHA-256(verificador)); verificador de 86 caracteres.	Conforme; verificadores não se repetem entre execuções.	Aprovado
T114	Android Keystore	Parâmetro <i>state</i> do OAuth	O <i>state</i> é de uso único: a segunda leitura devolve vazio (anti-replay).	—	Pendente
T115	Spotify Web API	Chamada real em rede, conta Premium	O comando é efetivamente executado no app Spotify.	—	Pendente

4.3 Teste de usabilidade · Heurísticas de Nielsen

Avalie a interface desenvolvida considerando as 10 heurísticas de usabilidade de Jakob Nielsen.

A avaliação foi conduzida sobre as cinco telas implementadas — Login, Início, Calibração, Configurar Gestos e Rastreamento Ativo. Vale registrar que o público-alvo do Nodify tem limitação motora, o que dá peso especial às heurísticas 3 (controle e liberdade) e 7 (flexibilidade), já que o usuário pode não conseguir corrigir um erro com um toque rápido na tela.

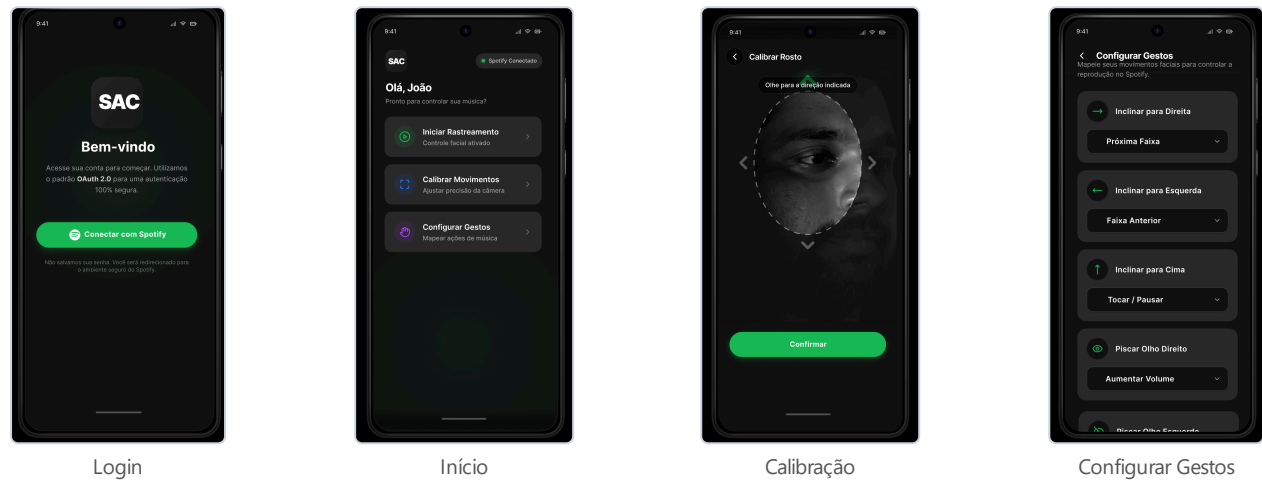


Figura 4 — Telas avaliadas (protótipo de alta fidelidade que orientou a implementação). O protótipo ainda exibe a marca anterior no logo; a renomeação para Nodify já está aplicada no aplicativo.

Heurística	Atende?	Evidência / Justificativa
1. Visibilidade do status do sistema	Sim	Indicador de carregamento na busca da faixa; selo “Spotify Conectado” na Home; ponto pulsante “Rastreamento Ativo”; pílula que exibe o gesto reconhecido e a ação disparada; botão “Salvar” mostra estado de gravação. Na calibração, o botão “Confirmar” só habilita quando o usuário atinge o limiar.
2. Correspondência entre sistema e mundo real	Sim	Todo o vocabulário está em português natural — “Inclinar para Direita”, “Aceno (sim)”, “Próxima Faixa” — sem jargão técnico (<i>roll</i> , <i>pitch</i> , <i>yaw</i> não aparecem na interface). Os ícones de seta acompanham a direção física do gesto.
3. Controle e liberdade do usuário	Parcial	Há “Voltar” na configuração e “Parar Rastreamento” sempre visível. Porém não existe desfazer após salvar um mapeamento: a única saída é “Restaurar padrões”, que descarta <i>todas</i> as personalizações de uma vez (ver PU01).
4. Consistência e padrões	Sim	Cores e tipografia vêm de um arquivo único de <i>design tokens</i> ; botões primários têm sempre a mesma altura (56 dp) e o mesmo formato; ícones direcionais usam a variante espelhada automaticamente em idiomas RTL.
5. Prevenção de erros	Parcial	O volume é saturado em 0–100 antes do envio; o botão “Salvar” é desabilitado durante a gravação; a tela de login ignora toques repetidos enquanto autentica (evita duas trocas de código OAuth); há intervalo mínimo de 1 s entre gestos, impedindo disparo duplo. Falta confirmação antes de “Restaurar padrões”, que é destrutivo (ver PU01).
6. Reconhecimento em vez de memorização	Sim	A tela de configuração lista os 9 gestos com nome, ícone e a ação vigente visível — o usuário reconhece as opções em vez de lembrá-las. As ações disponíveis aparecem numa lista, não em campo livre.

Heurística	Atende?	Evidência / Justificativa
7. Flexibilidade e eficiência de uso	Sim	O mapeamento gesto→ação é totalmente personalizável, permitindo que cada usuário use apenas os movimentos que consegue executar. A tela de rastreamento mantém botões manuais como caminho alternativo aos gestos.
8. Design estético e minimalista	Sim	Tema escuro com hierarquia tipográfica clara; cada tela apresenta um número reduzido de ações; a tela de rastreamento prioriza o feed da câmera e mantém os controles num painel inferior compacto.
9. Reconhecimento, diagnóstico e recuperação de erros	Parcial	As falhas mais prováveis têm mensagem específica e acionável (“Spotify Premium necessário...”, “Nenhum dispositivo ativo. Abra o Spotify no celular primeiro.”) em vez de código HTTP. Entretanto, a falha ao buscar a faixa atual na tela inicial é silenciosa (ver PU02).
10. Ajuda e documentação	Não	Não há tela de ajuda, tutorial inicial nem explicação de que o Spotify precisa estar aberto e ativo — e de que alguns comandos exigem conta Premium. O usuário só descobre a exigência ao receber a mensagem de erro (ver PU03).

Problemas de usabilidade encontrados

ID	Problema identificado	Heurística relacionada	Correção
PU01	“Restaurar padrões” apaga todas as personalizações sem pedir confirmação e sem oferecer desfazer. Um toque acidental descarta toda a configuração do usuário.	5. Prevenção de erros 3. Controle e liberdade	Proposta: exibir diálogo de confirmação (“Isto vai apagar seus gestos personalizados. Continuar?”) antes de executar.
PU02	Se a consulta da faixa atual falha na tela inicial, o erro é descartado silenciosamente: a área de “Tocando agora” simplesmente não aparece, sem informar por quê.	9. Reconhecimento e recuperação de erros 1. Visibilidade do status	Proposta: tratar também o caminho de falha, exibindo aviso discreto de indisponibilidade com opção de tentar novamente.
PU03	Não há orientação inicial sobre os pré-requisitos do sistema (Spotify aberto e ativo; conta Premium para controle de reprodução; boa iluminação para a câmera).	10. Ajuda e documentação 5. Prevenção de erros	Proposta: incluir tela de boas-vindas na primeira execução e um ícone de ajuda na tela de rastreamento.

As três correções estão **propostas, não implementadas**. Nenhum código de produção foi alterado durante a elaboração deste relatório — os testes foram escritos sobre a implementação existente, justamente para que os problemas encontrados fossem registrados antes de qualquer ajuste.

Verificações automatizadas de acessibilidade

Além da avaliação heurística, foram implementados 8 testes que percorrem a árvore de acessibilidade das telas e verificam duas regras objetivas, ambas ligadas ao público-alvo:

- todo elemento clicável expõe um rótulo para leitor de tela (sem isso o TalkBack anuncia apenas “botão”, tornando a navegação por voz inútil);
- todo alvo de toque mede ao menos 48 dp × 48 dp, conforme a diretriz do Material Design.

Esses testes integram o conjunto instrumentado e estão pendentes de execução em dispositivo. Aspectos não automatizáveis — contraste percebido, ordem lógica de foco e tempo real de navegação com leitor de tela ligado — estão previstos no roteiro de teste com usuários documentado no projeto.

4.4 Seleção das ferramentas de teste

Justificativa da escolha de ferramentas por nível de teste, incluindo as alternativas avaliadas e descartadas.

O critério que guiou todas as decisões foi o mesmo: **preferir a ferramenta já presente no grafo de dependências e que produza a evidência mais forte**. Cada biblioteca nova precisa ser defendida, e “roda no Android de verdade” é um argumento melhor do que “roda mais rápido”. O conjunto completo foi construído sem adicionar nenhuma dependência de teste ao projeto.

Nível	Escolhido	Descartado	Por quê
Unitário	JUnit 4	JUnit 5 (Jupiter)	O runner instrumentado (AndroidJUnit4) é construído sobre o JUnit 4. Adotar o 5 exigiria plugin Gradle de terceiros e deixaria os dois conjuntos de teste em frameworks diferentes.
Unitário	MockK	Mockito	Classes Kotlin são <code>final</code> por padrão e o Mockito precisaria do <i>mock-maker-inline</i> . O MockK também simula função <code>suspend</code> sem configuração extra — aqui isso é a regra, não a exceção.
Unitário	kotlinx-coroutines-test	Espera real (<code>Thread.sleep</code>)	Dois ViewModels fazem consulta periódica infinita. Com tempo virtual o ciclo avança em microssegundos; com espera real cada teste levaria segundos e ficaria instável.
Unitário	StateFlow.value	Turbine	Todos os fluxos expostos são StateFlow, que já carregam o valor corrente. O Turbine resolveria fluxo frio, que o projeto não usa — seria dependência nova sem problema correspondente.
Integração	Dispositivo real	Robolectric	O armazenamento cifrado depende do Android Keystore, apoiado em hardware. Simular na JVM testaria o simulador, não a cifragem — e a cifragem é justamente a regra que precisa de prova.
Integração	Room em memória	Banco em arquivo	Isola cada teste sem resíduo da execução anterior e dispensa rotina de limpeza.
Funcional	Compose UI Test	Espresso	A interface é 100 % Compose; os seletores do Espresso não enxergam nós do Compose.
Funcional	createComposeRule	createAndroidComposeRule	As telas recebem estado e ações por parâmetro, sem conhecer ViewModel. Sem Activity e sem injeção de dependência, o teste fica mais rápido e isola a tela.
Funcional	—	UI Automator	Só seria necessário para verificar a entrega do comando ao aplicativo do Spotify. O MVP fala com a Web API, então esse salto entre apps está fora de escopo.
Usabilidade	Heurísticas de Nielsen	Apenas teste com usuário	A avaliação heurística encontra problemas sem depender de recrutar o público-alvo, que é difícil de recrutar. Roda antes e barateia a sessão com usuário real.
Usabilidade	Sessão moderada + SUS	Teste remoto não moderado	Latência percebida e esforço motor precisam ser observados. O SUS ainda entrega um número comparável (média de referência: 68).

4.5 Melhorias identificadas

Oportunidades de melhoria levantadas pelos quatro níveis de teste, ordenadas pelo custo de não corrigir.

ID	Melhoria	Origem	Prioridade
M08	Exclusão dos dados de calibração pelo aplicativo. O projeto declara o direito de eliminação (LGPD) como regra inegociável e o roteiro de usabilidade já prevê a tarefa T6 para exercitá-lo, mas a função não existe: há apenas limpeza interna de tokens, sem caminho na interface.	Requisito × código	Alta
M01	Diálogo de confirmação antes de “Restaurar padrões”, que hoje descarta todas as personalizações num toque, sem desfazer.	Heurísticas 3 e 5	Alta
M07	Executar os 84 testes instrumentados. Estão implementados e compilam, mas nunca rodaram por falta de dispositivo ou emulador.	Cobertura	Alta
M02	Tratar o caminho de falha ao buscar a faixa atual, hoje descartado em silêncio.	Heurística 9	Média
M03	Onboarding com os pré-requisitos: Spotify aberto e ativo, conta Premium para controle e boa iluminação.	Heurística 10	Média
M04	Extrair uma sessão de calibração pura para a camada de domínio, tornando sua máquina de estados testável na JVM.	Lacuna de cobertura	Média
M09	Levar o feedback de iluminação para a tela de rastreamento — hoje só existe na calibração, mas a luz cai igual durante o uso.	Cobertura	Média
M05	Cobrir o interceptador de autenticação (401 → renovação, 429 → espera); exigiria MockWebServer.	Lacuna de cobertura	Baixa
M06	Configurar medição de cobertura de código para substituir a contagem manual por classe.	Lacuna de cobertura	Baixa

O achado mais relevante é o M08, e ele não veio de um teste que falhou: veio do cruzamento entre o roteiro de teste de usabilidade e o código. A tarefa T6 do roteiro exercita o direito de eliminação previsto na LGPD, e ao preparar sua execução constatou-se que a funcionalidade nunca foi implementada — nenhum teste automatizado poderia tê-la reprovado, porque não havia o que reprovar. É um argumento concreto de por que os quatro níveis não se substituem: cada um enxerga uma classe de problema que os outros não alcançam.